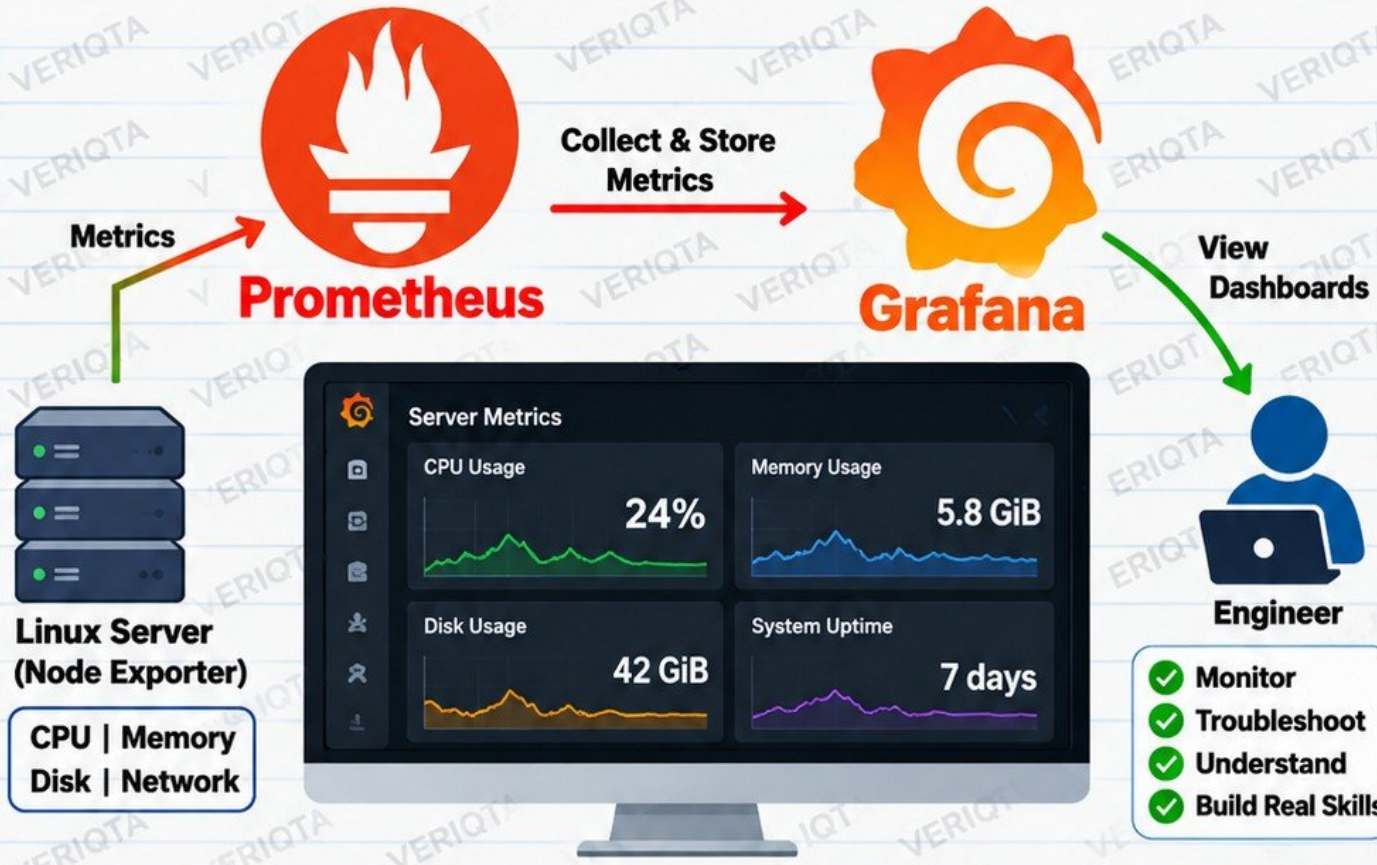
@veriqta

Prometheus and Grafana for Beginners:

Your First Dashboard



Real Examples | Hands-On Labs | Practical Queries | Complete Dashboard

VERIQTA

Build Real DevOps Skills

Your Server Is Running, But Is It Healthy?

1. A Real Engineering Problem

Your **application is slow**. Users are complaining that pages take a long time to load. You check the server and it is **online**. Ping works. SSH works. The website is still reachable. **So what is the problem?**

Common Situation

- Server is running
- Website is reachable
- Users still experience slow performance
- No clear error messages

2. Online Does Not Mean Healthy

A server can be online but still have serious problems that affect application performance. The server may be experiencing high **CPU usage, low memory, disk pressure,** or **too many requests**. These problems do not always make the server go down, but they make the application slow and unreliable.

Key Point

A running server only means it accepts connections. **It does NOT mean the application is** performing well or that users are having a good experience.

Healthy Server



- CPU: 15%
- Memory: 40%
- Disk: 30%
- Network: Normal

Application responds quickly
Users have a good experience

VS

Server Under CPU Pressure



- CPU: 95%
- Memory: 80%
- Disk: 70%
- Network: High

Server is still online
Application is slow
Users experience delays

3. What Is Monitoring?

Monitoring means **collecting and tracking data** (called **metrics**) from your servers and applications. Metrics show you what is happening inside your systems so you can **find problems early**, understand their impact, and keep your services reliable.

Examples of Metrics

- CPU usage
- Memory usage
- Disk space
- Network traffic
- Application metrics
- Response time
- Active users
- Error rate

4. Questions Engineers Must Answer

1. Is the server reachable?
2. Are there resource bottlenecks (CPU, memory, disk)?
3. Is the application responding normally?
4. When did the problem start?
5. Is the issue affecting one server or multiple servers?
6. What changed recently?

Junior Engineer Insight

Availability and performance are different things.

- Availability means the server is online and reachable.
- Performance means the application is working well and responding quickly.
- A server can be available but still have poor performance.
- Monitoring helps you understand both.

Prometheus vs. Grafana: Who Does What?

1. The Big Picture

Prometheus collects and stores metrics from your servers and applications. **Grafana** queries a data source (like Prometheus) and displays the metrics in visual dashboards.

Key Idea

Prometheus collects data.
Grafana visualizes data.

2. How They Work Together

Linux Server (Node Exporter)



- Exposes system metrics
- CPU, memory, disk
- Metrics endpoint on port 9100

Metrics (HTTP) →

Prometheus

- Scrapes metrics
- Stores time-series data
- Port 9090
- You query it using PromQL

Query (PromQL) →

Grafana

- Queries Prometheus
- Displays data in dashboards
- Create graphs, panels
- Port 3000

View Dashboard →

Engineer



- Views dashboards
- Monitors health
- Investigates issues
- Makes decisions based on data

3. Responsibilities Compared

Prometheus (Data Collection & Storage)

- Collects metrics from configured targets
- Pulls metrics at regular intervals (scraping)
- Stores metrics as time-series data
- Uses PromQL for querying
- Designed for reliability and long-term storage
- Alerts can be configured (optional)

Grafana (Visualization & Dashboards)

- Queries a data source (e.g. Prometheus)
- Turns metrics into visual dashboards
- Provides graphs, charts, tables, and alerts
- Helps you explore and analyze data
- Supports multiple data sources
- Does not collect or store metrics

4. Important Things to Know

1. Prometheus pulls metrics from targets. It does not receive data unless a target exposes metrics (e.g. Node Exporter).
2. Grafana does not collect metrics. It only displays data from a data source.
3. You can use Grafana with other data sources (like Loki, InfluxDB), but in this guide we will use Prometheus.
4. Node Exporter is just one exporter. There are many exporters for different services (Nginx, MySQL, Docker, Kubernetes, etc.).
5. All three tools work together to give you visibility into your systems.

Common Beginner Mistake

Assuming Grafana automatically collects server metrics.

Grafana does not collect metrics on its own. If Prometheus is not configured or not scraping your server, Grafana will have no data to display.

Junior Engineer Insight

Think of Prometheus as the data collector and storage system, and Grafana as the visualization and analysis tool. Both are needed.

VERIQTA

How a Metric Becomes a Dashboard Graph

1. The Complete Journey

Let's follow one CPU metric from a Linux server all the way to a Grafana graph.
This shows how metrics are collected, stored, queried, and visualized.

1. Linux Server



- Runs an application
- Generates system metrics (CPU, memory, disk, etc.)

2. Node Exporter



- Collects Linux metrics
- Exposes metrics via HTTP endpoint
- Runs on port 9100

3. Prometheus



- Scrapes metrics at regular intervals
- Stores time-series data
- Port 9090

4. Grafana



- Queries Prometheus using PromQL
- Displays data in dashboards
- Port 3000

5. Engineer



- Views graphs
- Monitors health
- Investigates issues
- Makes decisions based on data

2. Example Metric: CPU Usage

Raw metric from Node Exporter

http://localhost:9100/metrics

```
node_cpu_seconds_total{
  cpu="0",
  mode="idle"
} 12345.67
```

- This is a counter that keeps increasing.
- It shows total time the CPU spent idle.
- It includes labels like cpu and mode.

Stored in Prometheus

Timestamp	Value
10:00:15	12345.67
10:00:30	12347.12
10:00:45	12348.51
10:01:00	12350.03

- Prometheus scrapes the endpoint every 15 seconds (or your configured interval).
- Each value is stored with a timestamp.

Queried in Grafana (PromQL)

```
100 * (1 - avg(
  rate(node_cpu_seconds_total{
    mode="idle"
  }[5m]))
))
```

- rate() calculates the per-second change.
- We subtract idle time from 1 to get CPU usage.
- We multiply by 100 to show a percentage.

3. The Result: Grafana Graph



What the graph shows

- Each point represents a time and value.
- The graph is based on the data stored in Prometheus.
- You can change the time range (e.g. last 5 minutes, 1 hour).
- Grafana only displays what is available in the data source.

Key Lesson

A dashboard can only display data that is available from its data source. Grafana does not collect server metrics. It queries Prometheus and visualizes the data that Prometheus has already collected and stored.

Common Beginner Mistake

Assuming Grafana automatically collects server metrics.

Grafana is only a visualization tool. If Prometheus is not scraping your server, Grafana will have no data to display.

Junior Engineer Insight

Metrics are collected repeatedly over time. This allows you to see trends, detect problems early, and understand how your system behaves.

Metrics, Labels, and Time Series Explained

1. What Is a Metric?

A metric is a numeric measurement of something in your system. It has a **name**, a **value**, a **timestamp**, and optional **labels**.

Key Idea

A metric records how your system behaves over time. Monitoring uses these metrics to detect problems, understand trends, and make decisions.

2. Anatomy of a Metric

```
node_cpu_seconds_total{cpu="0",mode="idle",job="node",instance="node1"}
```

Metric name

Identifies what is being measured. This shows the total CPU time spent in different modes.

Labels (key=value)

- `cpu="0"` → CPU core number
- `mode="idle"` → CPU mode
- `job="node"` → Target job name
- `instance="node1"` → Server name

Curly braces

Labels are inside { }. They add context and help distinguish different time series.

3. Numeric Samples and Timestamps

Timestamp	Value
2026-10-18 10:00:00	12345.67
2026-10-18 10:00:15	12346.12
2026-10-18 10:00:30	12347.01
2026-10-18 10:00:45	12348.20

- Each sample has a timestamp and a numeric value.
- Prometheus collects a new sample at regular intervals (e.g. every 15 seconds).
- Over time, these samples form a time series.

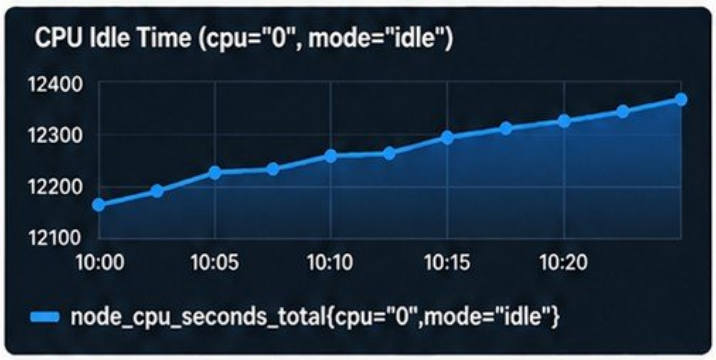
4. Labels Create Different Series

Same metric name, different labels = different time series

```
node_cpu_seconds_total{cpu="0",mode="idle"}   Series 1
node_cpu_seconds_total{cpu="1",mode="idle"}   Series 2
node_cpu_seconds_total{cpu="0",mode="user"}   Series 3
```

Each unique combination of labels (cpu, mode, job, instance, etc.) creates a separate time series. This allows you to filter and compare data.

5. How This Looks in a Graph



6. Why Labels Are Important

- Help you filter and query specific data
- Let you compare metrics (e.g. different CPUs or servers)
- Provide useful context (job, instance, environment)
- Enable powerful queries in PromQL
- Allow you to build flexible dashboards in Grafana

7. Too Many Labels = Problems

- Creates a huge number of time series
- Uses a lot of storage
- Makes Prometheus slower
- Increases memory and CPU usage
- Makes queries and dashboards difficult to manage

Production Awareness

Avoid using unique request IDs, user IDs, session IDs, or other high-cardinality values as labels. This can create millions of time series and cause performance and storage problems.

Practical Example

```
node_cpu_seconds_total{cpu="0",mode="idle"}
```

- Shows the total CPU time spent in idle mode for CPU core 0.
- You can change the labels to query other CPUs or modes.

05

Build Your First Monitoring Lab

1. What We Are Building

We will create a simple monitoring environment using Docker Compose. It will run three containers: Node Exporter, Prometheus, and Grafana. You will collect real metrics from your Linux host and view them in a Grafana dashboard.

What You Will Learn

- Run Prometheus, Grafana, and Node Exporter
- Understand what each tool does
- See how they connect using a private network
- Access the tools on ports 9090, 3000, and 9100
- Verify that metrics are being collected
- Build a simple monitoring lab you can reuse

2. Lab Architecture

Docker Network (monitoring)

Node Exporter



Port 9100

- Exposes Linux system metrics
- Runs on your host
- Provides /metrics endpoint

Metrics (HTTP) →

Prometheus



Port 9090

- Scrapes metrics from Node Exporter
- Stores time-series data
- You query it using PromQL

Query (PromQL) →

Grafana



Port 3000

- Connects to Prometheus
- Displays metrics in dashboards
- Create graphs, panels, and alerts
- You view and analyze the data

3. Required Software

- **Docker** (version 20.10 or later) Runs the containers
- **Docker Compose** (version 2 or later) Defines and starts the lab
- **Linux host** Any recent Linux distribution (Ubuntu, Debian, Rocky Linux, etc.)

4. Container Ports

Container	Port	What It Is Used For	Access URL (Local)
Node Exporter	9100	Exposes system metrics	http://localhost:9100/metrics
Prometheus	9090	Prometheus web UI	http://localhost:9090
Grafana	3000	Grafana web UI	http://localhost:3000

5. Docker Compose File

docker-compose.yml

```yaml
version: "3.8"
services:
  prometheus:
    image: prom/prometheus:latest
    container_name: prometheus
    ports:
      - "9090:9090"
    volumes:
      - ./prometheus/prometheus.yml:/etc/prometheus/prometheus.yml
    networks:
      - monitoring
  grafana:
    image: grafana/grafana:latest
    container_name: grafana
    ports:
      - "9000:3000"
    networks:
      - monitoring
  node-exporter:
    image: prom/node-exporter:latest
    container_name: node-exporter
    ports:
      - "9100:9100"
    pid: "host"
    networks:
      - monitoring
networks:
  monitoring:
    driver: bridge
```

6. Start the Lab

```
# Create a project directory
mkdir monitoring-lab
cd monitoring-lab

# Save the docker-compose.yml file

# Start the containers
docker compose up -d

# Check running containers
docker compose ps
```

7. Verify It Works

1. Open http://localhost:9090 You should see the Prometheus UI.
2. Open http://localhost:3000 Log in with admin / admin and change the password.
3. Open http://localhost:9100/metrics You should see raw metrics from Node Exporter.

Security Reminder

- Bind to localhost (127.0.0.1) when possible.
- Do not expose these ports publicly on the internet.
- This lab is for learning. Use a firewall if needed.
- In production, secure access with authentication, a reverse proxy, and network rules.

Key Takeaway

All three containers run on a private Docker network. Node Exporter exposes metrics, Prometheus collects and stores them, and Grafana queries Prometheus to display the data. A dashboard can only show data that is available in its data source.

VERIQTA

Start Prometheus and Check Its Health

What You Will Learn

- Understand the key parts of prometheus.yml
- Start Prometheus using Docker Compose
- Check that Prometheus is running and ready
- Open the Prometheus web interface
- Verify that targets are being scraped

1. Key Parts of prometheus.yml

This file tells Prometheus what to scrape and how often.

prometheus.yml

```yaml
global:
  scrape_interval: 15s     # how often to collect metrics
  evaluation_interval: 15s  # how often to evaluate rules

scrape_configs:
  # Prometheus self-monitoring
  - job_name: 'prometheus'
    static_configs:
      - targets: ['prometheus:9090']  # scrape itself

  # Node Exporter (Linux host metrics)
  - job_name: 'node'
    static_configs:
      - targets: ['node-exporter:9100']
```

2. What These Settings Mean

Setting	What It Does
scrape_interval	How often Prometheus collects metrics from targets (e.g. every 15 seconds).
evaluation_interval	How often Prometheus evaluates rules (e.g. alerting rules).
scrape_configs	List of targets to scrape.
job_name	A name for the target (e.g. prometheus, node).
static_configs	Defines the target addresses.
targets	The hostname and port of the metrics endpoint.

3. Start Prometheus (and the whole lab)

From tile directory that contains docker-compose.yml, run:

```
# Start all containers
docker compose up -d

# Check running containers
docker compose ps
```

4. Check That Prometheus Is Running

Run the command below and look for the prometheus container.

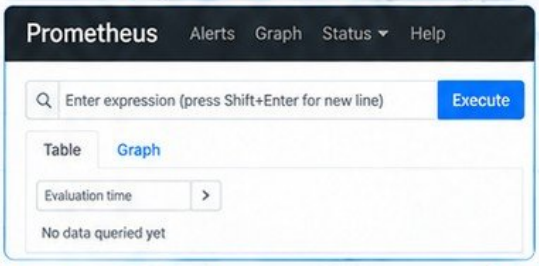
```
docker compose ps
```

NAME	IMAGE	STATUS	PORTS
prometheus	prom/prometheus:latest	Up	0.0.0.0:9090->9090/tcp
grafana	grafana/grafana:latest	Up	0.0.0.0:3000->3000/tcp
node-exporter	prom/node-exporter:latest	Up	0.0.0.0:9100->9100/tcp

5. Open the Prometheus Interface

Open your browser and go to:

http://localhost:9090



6. Check Prometheus Readiness

- You should see the Prometheus web UI.
- Go to Status → Targets.
- You should see your targets listed (prometheus and node).
- The state should be UP if everything is working.



7. Practice

- Run docker compose ps.
- Confirm the prometheus container is Up.
- Open http://localhost:9090 and verify the UI loads.
- Check Status → Targets and confirm both targets are UP.

8. Troubleshooting

Container is stopped

- docker compose ps shows the container is not running.
- Start it with docker compose up -d.
- Check for errors with docker compose logs prometheus.

Target is DOWN (failed scrape)

- Container is running, but target shows DOWN.
- Check the target URL and port.
- Make sure the target container is running.
- Check the docker network and container name.
- Look at Prometheus logs for scrape errors.

Security Reminder

- Bind interfaces to localhost (127.0.0.1) when possible.
- Do not expose ports 9090, 3000, and 9100 publicly on the internet.
- This lab is for learning. Use a firewall if needed.

Key Takeaway

Prometheus is working when the container is running, the UI is accessible, and your targets are UP. A stopped container and a failed scrape are different problems and require different troubleshooting steps.

VERIQTA

VERIQTA @veriqta

07

Node Exporter: Give Your Linux Server a Voice

1. What Is an Exporter?

An exporter **collects metrics** from a system or application and **exposes** them in a format that **Prometheus** can **scrape**. It gives your Linux server a voice by turning system information into metrics.

2. How Node Exporter Works

- Reads system metrics from the Linux host (CPU, memory, disk, network, etc.).
- Exposes the metrics via an HTTP endpoint.
- Runs as a simple, lightweight service.
- Prometheus scrapes this endpoint at regular intervals.

Key Idea

Node Exporter does not store metrics. It only exposes them. **Prometheus** collects and stores the metrics, and **Grafana** visualizes them.

3. Node Exporter in the Monitoring Lab

Linux Host



- CPU, memory, disk,
- network, filesystem and more

System metrics →

Node Exporter



- Collects Linux metrics
- Exposes them at http://:9100/metrics
- Runs in a container

Metrics (HTTP) →

Prometheus



- Scrapes the /metrics endpoint
- Stores time-series data
- Queries using PromQL

Data for graphs →

Grafana



- Queries Prometheus
- Displays graphs and dashboards
- Helps you analyze your server metrics

4. The /metrics Endpoint

Open your browser and go to:

http://localhost:9100/metrics

```
# HELP node_cpu_seconds_total Total user,
system, idle and iowait  time spent in seconds.
# TYPE node_cpu_seconds_total counter
node_cpu_seconds_total{cpu="0",mode="idle"} 12345.67
node_cpu_seconds_total{cpu="0",mode="user"} 234.12
node_memory_MemAvailable_bytes 6319974400
node_filesystem_avail_bytes{mountpoint="/"} 45298401280
...
```

- This endpoint returns plain text metrics in a format that Prometheus can scrape.
- You will see many metrics, each with labels and values.
- The numbers keep increasing (for counters) or change (over time).

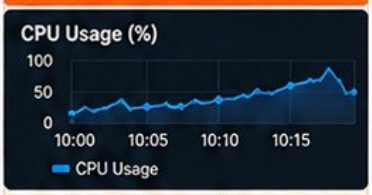
5. Raw Metrics vs. Dashboard Graphs

Raw Exported Metrics (/metrics endpoint)

```
node_cpu_seconds_total{cpu="0",mode="idle"} 12345.67
node_cpu_seconds_total{cpu="0",mode="user"} 234.12
node_memory_MemAvailable_bytes 6319974400
node_filesystem_avail_bytes{mountpoint="/"} 45298401280
...
```

- Plain text format
- Shows raw metric names, labels, and values
- Not visually appealing
- Used by Prometheus to collect data

Grafana Dashboard Graph



- Shows the same data as a graph
- Easy to understand
- Helps you see trends over time
- Queries data from Prometheus using PromQL

6. Practice: Inspect the Metrics

1. Open http://localhost:9100/metrics in your browser.
2. Look for metrics like node_cpu_seconds_total, node_memory_MemAvailable_bytes, and node_filesystem_avail_bytes.
3. Compare the values over time (refresh the page).
4. Notice the labels such as cpu, mode, instance, and job.

7. Important: Container Access to Host Metrics

When running Node Exporter in a container, it needs access to the host system (e.g. /proc, /sys, /, etc.) to read real Linux metrics. Without the correct access, the metrics may be incomplete or may show container metrics instead of host metrics.

Key Lesson

Containerized Node Exporter requires suitable host access to report Linux host metrics accurately. If it cannot access the host's /proc, /sys, and other system paths, the metrics may be incorrect or limited.

Common Issues

- Cannot access http://localhost:9100/metrics → Check if the container is running.
- Metrics show container data only → Mount the required host paths.
- No metrics or errors → Check container logs and permissions.

Quick Tips

- Node Exporter is lightweight and safe to run.
- It only exposes metrics; it does not change your system.
- Use it for monitoring Linux servers, VMs, or cloud instances.

VERIQTA

Scrape Targets: Is Prometheus Receiving Data?

08

Key Idea

- Prometheus discovers targets from its configuration.
- It scrapes each target at a regular interval (e.g. every 15 seconds).
- If the target responds, the metrics are stored.
- You can check the health of each target on the Status → Targets page.

1. How Prometheus Discovers Targets

Prometheus reads the scrape_configs section in prometheus.yml. For each target, it periodically sends an HTTP request to the /metrics endpoint and collects the data.

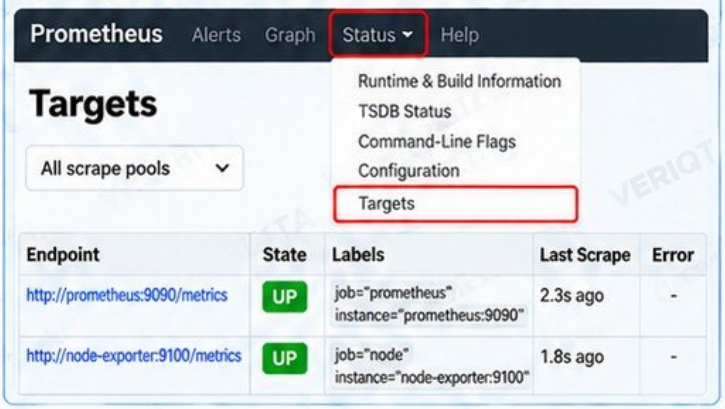
2. Our Targets in prometheus.yml

```yaml
scrape_configs:
  - job_name: 'prometheus'
    static_configs:
      - targets: ['prometheus:9090']

  - job_name: 'node'
    static_configs:
      - targets: ['node-exporter:9100']
```

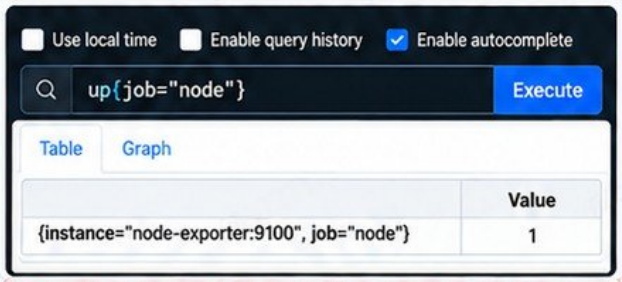
3. Target Health Page in Prometheus

Open http://localhost:9090 → Click Status → Targets.



4. The up Metric

The up metric shows whether a target is reachable.



- A value of 1 means the target is UP and metrics are being collected.
- A value of 0 means the target is DOWN and Prometheus cannot scrape it.

5. What UP and DOWN Mean

UP (Healthy)

- Prometheus successfully scraped the /metrics endpoint.
- Metrics are being collected and stored.
- You should see a value of 1 in the up metric.

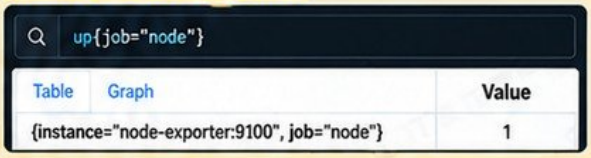
DOWN (Unhealthy)

- Prometheus could not scrape the target.
- Metrics are not being collected.
- You will see a value of 0 in the up metric.
- Check the error message on the Targets page.

6. Practice: Check Your Targets

1. Open http://localhost:9090 and go to Status → Targets.
2. Confirm that both prometheus and node targets are UP.
3. Run the query up{job="node"} and confirm it returns 1.
4. Check the Last Scrape time. It should update every few seconds.
5. Click on the endpoint URL to see the raw metrics.

Example Query Result



7. Troubleshooting Common Issues

Problem	Possible Cause	How to Fix
Target shows DOWN	Wrong port	Verify the port (e.g. 9100 for Node Exporter).
Target shows DOWN	Container not running	Run docker compose ps and start the container.
Target shows DOWN	Wrong hostname	Use the correct container name (e.g. node-exporter).
Connection refused	Endpoint not accessible	Check container network and port configuration.
Target returns 404	Wrong metrics path	Use /metrics (not / or other paths).

Key Takeaway

If a target is UP, Prometheus is receiving data from that endpoint. If it is DOWN, investigate the connection, port, and configuration before checking the metrics.

Important

- Always check the Targets page first.
- Use the up metric to quickly confirm status.
- Read the error message for details.
- Ensure containers are running and reachable on the correct ports and hostnames.

VERIQTA

09

PromQL: Ask Your Monitoring System Questions

1. What Is PromQL?

PromQL (Prometheus Query Language) allows you to ask questions about your metrics. You can select metrics, filter by labels, and use functions to find useful information.

Key Idea

A PromQL query should answer a specific operational question (e.g. "Is my server up?", "How high is CPU usage?", "Are there errors?").

2. Basic Query Components

```
metric_name{label="value", label2="value"}
```

Metric selector
The name of the metric you want to query.

Label filters (optional)
Narrow down the results using labels such as job, instance, mode, etc.

3. Instant Query vs. Range Query

Instant Query

- Returns the most recent value for each matching time series.
- Used for current state (e.g. Is it up? What is the current CPU usage?).
- Example: up or up{job="node"}

Range Query

- Returns a range of values over time.
- Used for graphs, trends, and functions like rate(), increase(), etc.
- Example: node_cpu_seconds_total[5m]

4. Common Label Filters

Filter	Meaning
{job="node"}	Only series with job=node
{instance="node-exporter:9100"}	Only series from this instance
{mode="idle"}	Only series with mode=idle
{job!="node"}	Exclude series with job=node
{job=~"node|prometheus"}	Match multiple values (regex)

5. Run Your First Queries

1. Run up (shows the status of all targets)

up — Execute

Table | Graph

Metric	Value
{instance="prometheus:9090", job="prometheus"}	1
{instance="node-exporter:9100", job="node"}	1

2. Run up{job="node"} (shows only the node target)

up{job="node"} — Execute

Table | Graph

Metric	Value
{instance="node-exporter:9100", job="node"}	1

6. More Useful Example Queries

Query	What It Shows
up	Status of all targets (1 = up, 0 = down)
up{job="node"}	Status of the node target
node_cpu_seconds_total	Raw CPU counter metric
node_cpu_seconds_total {mode="idle"}	CPU idle time only
rate(node_cpu_seconds_total [5m])	CPU usage rate over 5 minutes
node_memory_MemAvailable_bytes	Available memory in bytes
rate(node_network_receive_bytes_total [5m])	Incoming network traffic rate

7. Understanding the Results

```
{instance="node-exporter:9100", job="node"}  1
```

Labels
Describes this time series (instance and job).

Value
The most recent value (1 means the target is up).

- Each row is a separate time series.
- The labels identify the series.
- The value is the metric's most recent sample.

8. Practice

1. Open http://localhost:9090 and go to the Graph tab.
2. Run the query up and check the results.
3. Run up{job="node"} and confirm it returns a value of 1.
4. Try node_cpu_seconds_total and explore the labels.
5. Experiment with a range query like rate(node_cpu_seconds_total[5m]).

9. Troubleshooting

- **No results**
  Check the metric name and labels.
- **Empty result**
  The target may be down or the label filter does not match.
- **Value is 0**
  The target is not reachable.
  Check the Status → Targets page.
- **Query error**
  Check for typos in the query.
  Use the correct metric names and labels.

Key Takeaway

PromQL helps you ask specific questions about your system. Start with simple queries and build from there.

Remember

A query should always answer a real operational question. Use the correct metric name and labels, and check your results.

VERIQTA

Counters, Gauges, and Why rate() Matters

1. Two Common Metric Types

Counter (cumulative)

- Always increases over time (never goes down).
- Resets to 0 when the service restarts.
- Used for things like CPU time, requests, errors, etc.
- Example: `node_cpu_seconds_total`

Gauge (up and down)

- Can increase or decrease.
- Shows the current value at a point in time.
- Used for things like memory usage, temperature, active connections, etc.
- Example: `node_memory_MemAvailable_bytes`

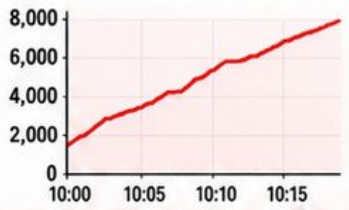
Key Idea

- Some metrics (counters) only increase.
- Some metrics (gauges) go up and down.
- You must use functions like `rate()` for counters to calculate a meaningful per-second value.

2. Visual Comparison

Counter Metric

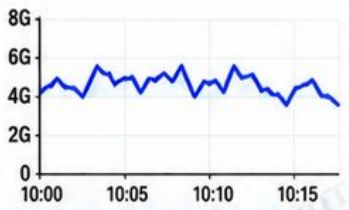
`node_cpu_seconds_total{mode="idle"}`



The value keeps increasing over time because it is a counter.

Gauge Metric

`node_memory_MemAvailable_bytes`



The value goes up and down because it is a gauge.

3. Why rate() Matters for Counters

The `node_cpu_seconds_total` metric is a **counter**. It shows the total number of seconds the CPU has spent in each mode since the system started. It always increases, so the raw value is not the current CPU usage. You use `rate()` to calculate how fast the counter is increasing over time (e.g. per second).

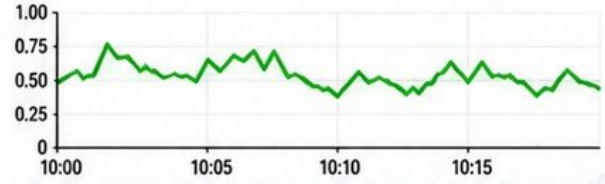
Analogy

A counter is like a car odometer. It always goes up. The speed (rate) tells you how fast the car is moving.

4. Example: CPU Idle Time Using rate()

```
rate(node_cpu_seconds_total{mode="idle"}[5m])   Execute
```

Table | Graph



This shows the per-second rate of increase for CPU idle time over the last 5 minutes.

5. Understanding the Query

```
rate(node_cpu_seconds_total{mode="idle"}[5m])
```

Function

`rate()` calculates how fast the counter is increasing per second.

Metric and Label Filter

`node_cpu_seconds_total{mode="idle"}` selects the CPU idle time counter.

Range

`[5m]` looks at the last 5 minutes of data.

6. Common Mistake

Treating cumulative CPU seconds as current CPU utilization.

If you query `node_cpu_seconds_total{mode="idle"}` directly, you will see a large, ever-increasing number. This is not the current CPU usage. You must use `rate()` (or `increase()`) to get a per-second value that shows how the CPU is being used.

7. More Examples

Query	What It Shows
`rate(node_cpu_seconds_total{mode="idle"}[5m])`	CPU idle rate over 5 minutes
`rate(node_cpu_seconds_total{mode="user"}[5m])`	CPU user mode rate over 5 minutes
`rate(node_cpu_seconds_total{mode="system"}[5m])`	CPU system mode rate over 5 minutes
`node_memory_MemAvailable_bytes`	Current available memory (gauge)

8. Key Takeaways

- Counters always increase.
- Gauges go up and down.
- Use `rate()` (or `increase()`) for counters.
- The raw counter value is not the current utilization.
- Always choose the right metric type and query for your operational question.

CPU, Memory, and Disk: The Metrics That Matter

1. Three Important Monitoring Questions

These three questions help you quickly understand the health of a Linux server:

1. **Is CPU busy?**
   (Is the server spending a lot of time doing work?)
2. **How much memory is available?**
   (Is the server running low on memory?)
3. **Is the filesystem running out of space?**
   (Is the disk filling up and close to full?)

2. The Key Metrics

Metric Name	Metric Type	Unit	What It Tells You
node_cpu_seconds_total	Counter	seconds	Total CPU time spent in each mode (user, system, idle, etc.). Always increases.
node_memory_MemAvailable_bytes	Gauge	bytes	Amount of memory available for new applications.
node_filesystem_avail_bytes	Gauge	bytes	Free space available on a filesystem.

3. Metric Examples and Graphs

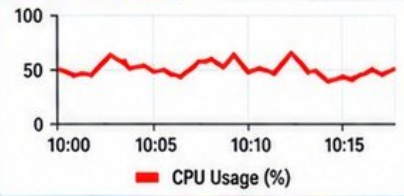
CPU Usage

Metric: node_cpu_seconds_total
Type: Counter
Unit: seconds
Meaning: Total time the CPU has spent in each mode. Use rate() to see per-second CPU usage.

Example Query (CPU usage %)

```
100 - (avg by(instance) (rate(node_cpu_seconds_total
{mode="idle"}[5m])) * 100)
```

Example Graph



- High CPU usage can indicate heavy load, inefficient applications, or performance problems.

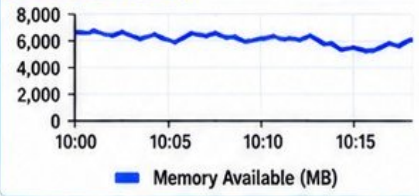
Memory Available

Metric: node_memory_MemAvailable_bytes
Type: Gauge
Unit: bytes (shown in MB/GB)
Meaning: Memory available for new applications (includes free memory, page cache, and reclaimable memory).

Example Query (Memory available)

```
node_memory_MemAvailable_bytes
/ 1024 / 1024
```

Example Graph



- High memory usage alone does not always mean a problem. Linux uses available memory for cache, which can be reclaimed when needed.

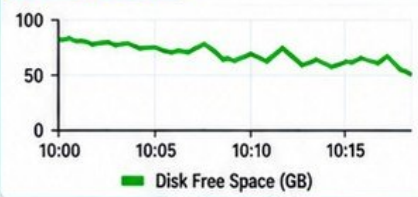
Disk Free Space

Metric: node_filesystem_avail_bytes
Type: Gauge
Unit: bytes (shown in GB)
Meaning: Free space available on a filesystem.

Example Query (Disk free space)

```
node_filesystem_avail_bytes{fstype!="tmpfs"}
/ 1024 / 1024 / 1024
```

Example Graph



- Low disk space can cause application errors, failed deployments, and system instability.

4. Practical PromQL Queries

Question	Query	What It Shows
CPU usage (%)	`100 - (avg by(instance) (rate(node_cpu_seconds_total {mode="idle"}[5m])) * 100)`	CPU utilization over time
Memory available (GB)	`node_memory_MemAvailable_bytes / 1024 / 1024 / 1024`	Available memory in GB
Disk free space (GB) (root filesystem)	`node_filesystem_avail_bytes{mountpoint="/"} / 1024 / 1024 / 1024`	Free space on / in GB

5. Key Takeaways

- Use rate() for counter metrics like node_cpu_seconds_total.
- Use gauge metrics like node_memory_MemAvailable_bytes and node_filesystem_avail_bytes.
- Monitor CPU, memory, and disk together to understand overall server health.
- High memory use alone does not prove memory exhaustion.

6. Common Mistakes

- Using node_cpu_seconds_total directly instead of rate().
- Assuming high memory usage always means a problem.
- Monitoring only one resource (CPU, memory, or disk).
- Ignoring filesystem type and monitoring temporary filesystems (e.g. tmpfs).

7. Production Insight

Look at trends over time, not just a single value. Short spikes are normal. Persistent high CPU, consistently low available memory, or steadily decreasing disk space require investigation.

VERIQTA

VERIQTA

Read a Graph Like an Engineer

Key Idea

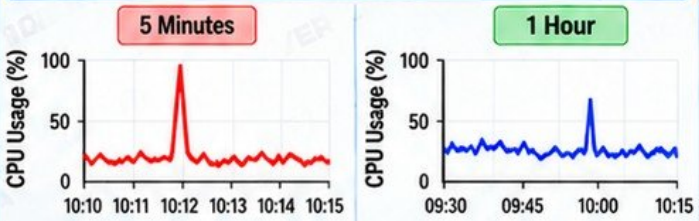
A graph tells a story about what is happening over time. Learn to look at the shape, units, time window, and how long a problem lasts, not just the highest value.

1. Key Things to Look For

What to Check	What It Means
Spikes	A short, sudden increase. May be normal if brief.
Trends	A steady increase or decrease over time.
Sustained high values	A problem if the metric stays high for a while.
Gaps	Missing data. Could be a scrape issue or target down.
Units	Know what the values mean (%, bytes, seconds, etc.).
Time window	A longer view shows the real pattern.

2. Understanding Time Windows

Same Data, Different Time Windows

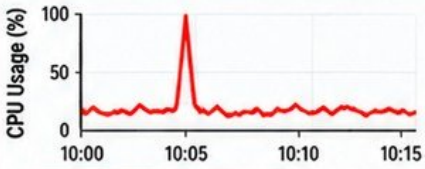


A short window shows a spike in more detail.

A longer window shows the overall trend and context.

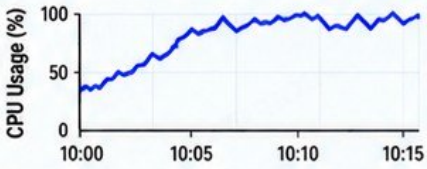
3. Three Common CPU Patterns

A. Short Spike (Usually OK)



- High CPU for a few seconds.
- Often caused by a short task (e.g. backup, deployment, or a batch job).
- May not require action if it quickly returns to normal.

B. Sustained High CPU (Investigate)



- CPU stays high for a sustained period.
- Can indicate heavy load, a runaway process, or insufficient resources.
- This pattern usually needs investigation.

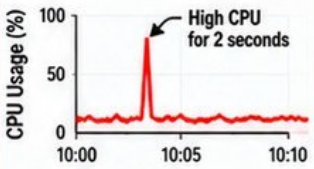
C. Gaps in Data (Check Health)



- Missing data points.
- May mean Prometheus is not scraping the target.
- Check target health, network, and configuration.

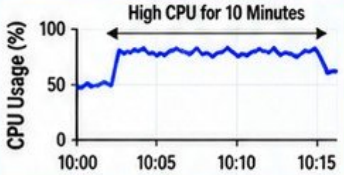
4. Why Duration Matters

High CPU for 2 Seconds



- Short spike.
- Could be a normal, quick task.
- Usually does not indicate a serious problem.

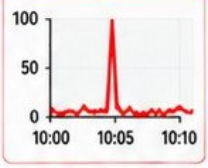
High CPU for 10 Minutes



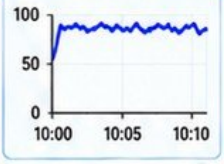
- Sustained high CPU.
- Indicates a real issue that may impact performance.
- Needs investigation to find the cause (e.g. a process, traffic, or resource limit).

5. Practice: Which Graph Needs Investigation?

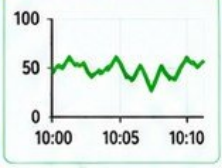
Graph A



Graph B



Graph C



Questions

1. Which graph deserves further investigation?
2. Why?
3. What could be the possible causes?

6. Common Mistakes

- Looking only at the highest value without checking the duration.
- Ignoring the time window (5 minutes vs 1 hour).
- Not checking the units.
- Assuming a short spike is a serious problem.
- Ignoring gaps in data.

7. Tips for Reading Graphs

- Always check the time window.
- Look at the overall trend, not just a single point.
- Understand the units.
- Compare with other related metrics.
- Investigate sustained high values and unusual patterns.

8. Key Takeaway

A good engineer looks at the whole graph, understands the context, and asks the right questions. A graph is most useful when you know what you are looking for.

VERIQTA

VERIQTA

Connect Grafana to Prometheus

Key Idea

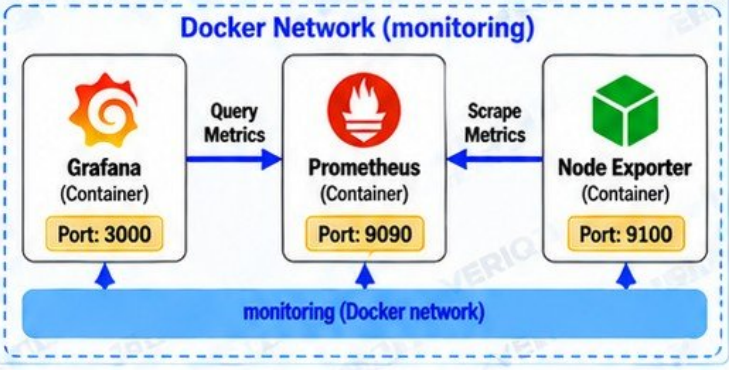
- A data source tells Grafana where to get data from.
- In our lab, Grafana communicates with Prometheus using the Docker network.
- Use the service name http://prometheus:9090 instead of localhost when Grafana runs in a container.

1. What Is a Data Source?

A data source in Grafana is a connection to a system that provides metrics, logs, or other data. In this lab, we add Prometheus as a data source so Grafana can query and visualize the metrics stored in Prometheus.

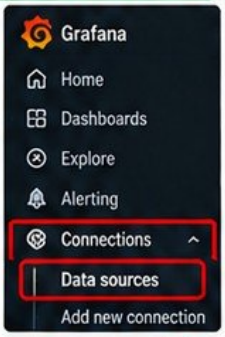
2. How Grafana Reaches Prometheus

Both Grafana and Prometheus run in the same Docker network. Grafana uses the service name prometheus and port 9090 to reach Prometheus.



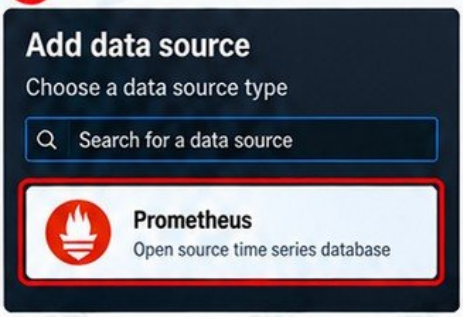
3. Open Grafana and Go to Data Sources

1. Open Grafana in your browser: http://localhost:3000
2. Log in (default is usually admin / admin in a local lab).
3. Go to Connections → Data sources in the left menu.



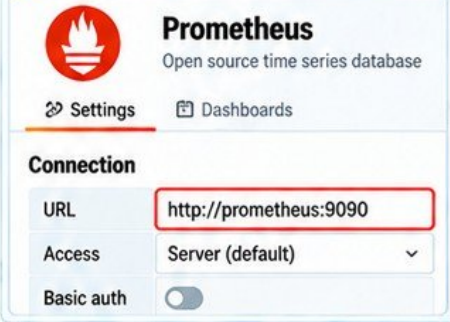
4. Add Prometheus as a Data Source

1. Select Prometheus



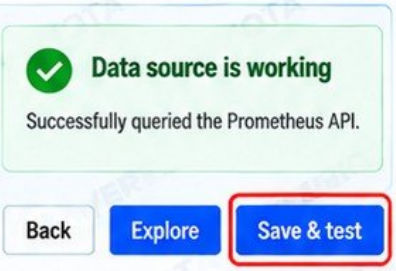
5. Configure the Connection

2. Set the URL and save.



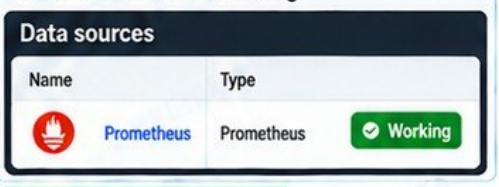
6. Test the Connection

3. Click Save & test.



7. Verify in Grafana

After saving, you should see the data source listed and marked as working.



8. Common Failure

Using localhost:9090 inside Grafana's container

- localhost inside the Grafana container refers to Grafana itself, not the Prometheus container.
- This will cause a connection error.
- Use http://prometheus:9090 (because prometheus is the service name on the Docker network).

9. Troubleshooting Tips

Problem	How to Fix
Connection refused	Check that the Prometheus container is running. Run: docker compose ps
Bad gateway / timeout	Use http://prometheus:9090 (not localhost).
Data source is not working	Check the Docker network and service name. Verify the URL and port.

10. Key Takeaway

Grafana needs the correct URL to reach Prometheus. In a Docker Compose lab, always use the service name http://prometheus:9090 instead of localhost:9090.

VERIQTA

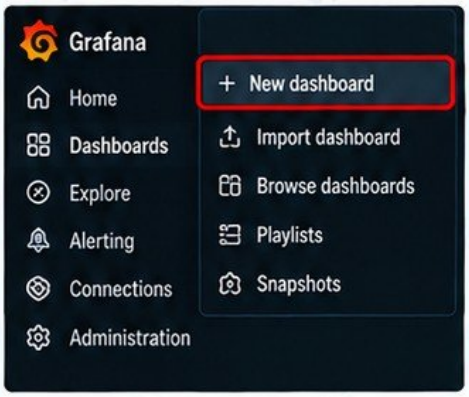
Create Your First Dashboard Panel

Key Idea

A dashboard should not just show numbers. It should clearly explain what the number means so anyone looking at it can understand the system's health.

1. Create a New Dashboard

1. Open Grafana (http://localhost:3000) and log in.
2. Click + → New dashboard.
3. Click **Add panel**.

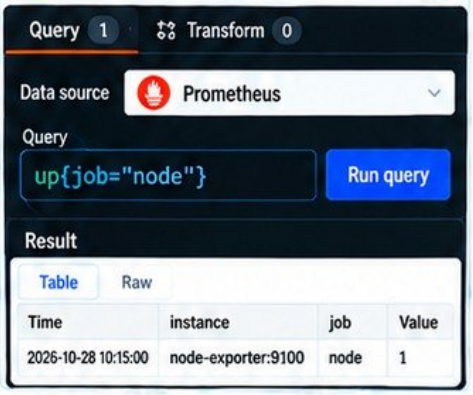


2. Set the Panel Query

1. Select your Prometheus data source.
2. Enter the query:

```
up{job="node"}
```

3. Run the query to check the result.

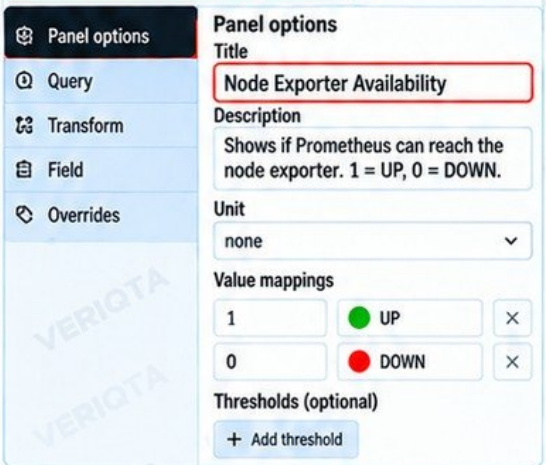


3. Choose Stat Visualization

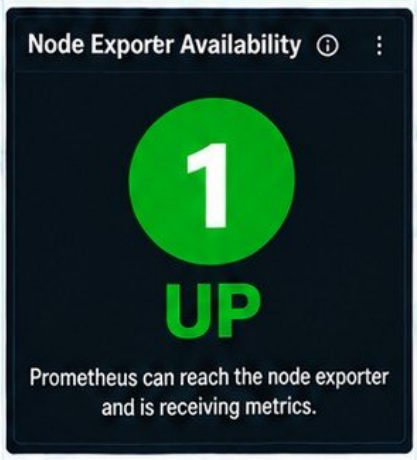
1. On the right, select **Stat** as the visualization type.
2. Configure the panel options (title, unit, and value mappings).



4. Configure the Panel



5. Stat Panel Preview

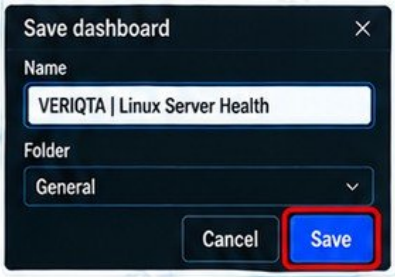


6. Save the Dashboard

1. Click **Apply** (top right).
2. Give it a name:

```
VERIQTA | Linux Server Health
```

3. Click **Save**.



7. What the Value Means

Value	Status	Meaning
1	UP	Prometheus is successfully scraping the node exporter.
0	DOWN	Prometheus cannot reach the node exporter. Check the target, network, and configuration.

8. Why This Panel Is Useful

- Quickly shows if your server is being monitored.
- Helps you detect problems early.
- Uses a clear title and description.
- Uses value mappings so the meaning is obvious.
- Can be combined with other panels (CPU, memory, disk) to build a complete dashboard.

9. Common Mistakes

- Showing 1 without explaining what it means.
- Using the wrong data source.
- Using localhost:9090 inside Grafana (instead of http://prometheus:9090).
- Not setting value mappings (1 = UP, 0 = DOWN).
- Using a confusing panel title.

Key Takeaway

A well-designed panel does more than show a number. It explains what the number means and helps you quickly understand the health of your system.

VERIQTA @veriqta

Build a Real CPU Utilization Graph

Key Idea

- We use the rate of CPU idle time, subtract it from 1 to get CPU usage, average across all CPU cores, and multiply by 100 to show a percentage.
- This gives a real, accurate view of overall host CPU utilization.

1. The PromQL Query

This query calculates the overall CPU utillization percentage for the host.

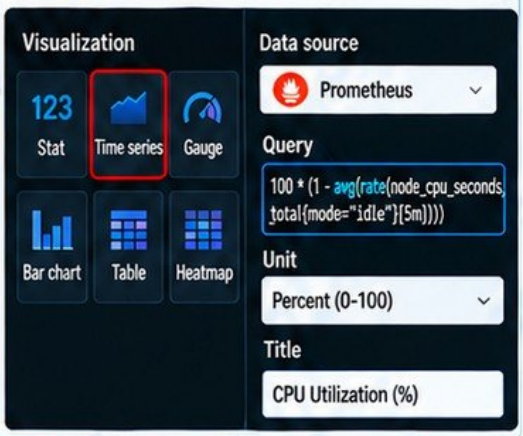
PromQL

```
100 * (1 - avg(rate(node_cpu_seconds_total{mode="idle"}[5m])))
```

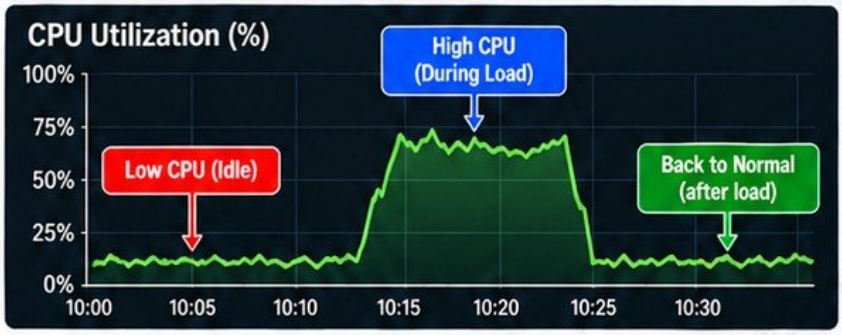
1. **rate(node_cpu_seconds_total{mode="idle"}[5m])**
   Calculates the per-second rate of increase for CPU idle time over 5 minutes.
2. **avg(...)**
   Averages the idle rate across all CPU cores.
3. **1 - ...**
   Subtracts the idle proportion from 1 to get the busy (used) proportion.
4. **100 * ...**
   Converts the result into a percentage.

2. Add a Time Series Panel in Grafana

1. Open your dashboard and click **Add panel.**
2. Select **Time series** as the visualization.
3. Choose your Prometheus data source.
4. Enter the PromQL query from step 1.
5. Set the panel title (e.g. **CPU Utilization (%)**).
6. Set the unit to Percent (0-100).
7. Click **Apply.**

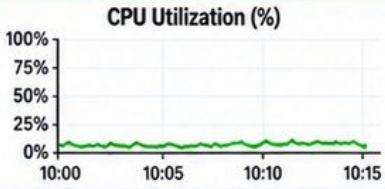


3. Example Graph in Grafana



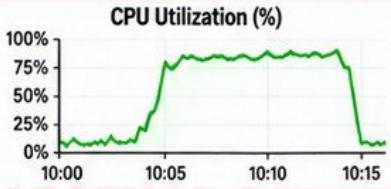
4. Compare Idle vs High CPU Activity

A. Idle (Low CPU Usage)



- The server is mostly idle.
- CPU utilization stays low (e.g. 1–10%).
- This is normal when there is no heavy workload.

B. Increased CPU Activity



- The server is under load.
- CPU utilization increases (e.g. 70–90%).
- This could be caused by a process, deployment, backup, or other workload.

5. Why This Query Works

1. **node_cpu_seconds_total{mode="idle"}**
   Gives the cumulative CPU idle time.
2. **rate(...[5m])**
   Calculates how fast the idle time is increasing over the last 5 minutes (as a per-second rate).
3. **avg(...)**
   Averages across all CPU cores so you get the overall host utilization, not just one core.
4. **1 - ...**
   Subtracts the idle proportion from 1 to get the busy (used) proportion.
5. **100 * ...**
   Converts the value to a human-friendly percentage (0–100%).

6. Practice

- View the graph when the server is idle.
- Run a CPU-intensive command (e.g. yes > /dev/null) in the server container to increase CPU activity.
- Observe how the graph changes.
- Stop the command and see the graph return to normal.

7. Common Mistakes

- Using node_cpu_seconds_total without rate().
- Forgetting to subtract from 1 (shows idle instead of usage).
- Not averaging across all cores.
- Not setting the unit to percent.
- Using a very short time window (too much noise).

8. Key Takeaway

This graph shows real CPU utilization by using the rate of idle time, averaging across cores, and converting to a percentage. It helps you quickly understand how busy your server is.

VERIQTA
@veriqta @veriqta @veriqta @veriqta @veriqta @veriqta

VERIQTA @veriqta

Add Memory and Disk Space Panels

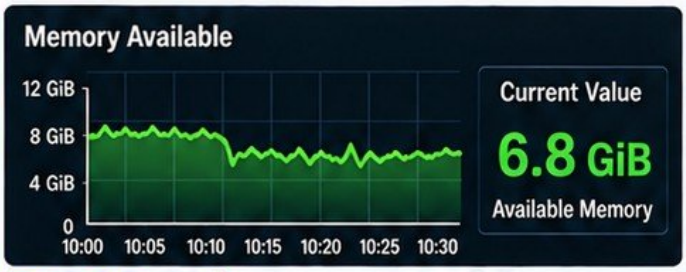
16

Key Idea

- A good dashboard shows the most important system resources: memory and disk space.
- Each panel should clearly identify what is being measured, show the unit, and make the information easy to understand.

1. Memory Availability Panel

We use `node_memory_MemAvailable_bytes` to show how much memory is available for new applications.



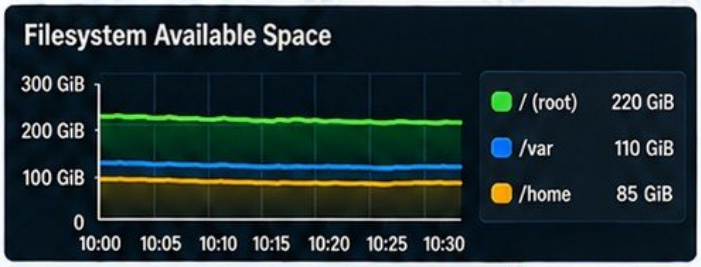
PromQL Query

```
node_memory_MemAvailable_bytes
```

- Shows the amount of memory available for new applications.
- Unit is bytes (we display it as GiB in Grafana).
- This includes free memory, page cache, and reclaimable memory.

2. Disk Space Panel

We use `node_filesystem_avail_bytes` to show the available space on a filesystem.



PromQL Query (example for the root filesystem)

```
node_filesystem_avail_bytes{mountpoint="/"}
```

- Shows the available disk space for the selected filesystem.
- Unit is bytes (we display it as GiB in Grafana).
- You can filter by mountpoint to show specific filesystems (e.g, /, /var).

3. Understanding Bytes, GiB, and Percentages

Unit	Meaning	Example	Grafana Display
Bytes	Base unit of storage	1073741824 bytes	May show as bytes or converted.
GiB	Gibibytes (1024^3 bytes)	1 GiB = 1,073,741,824 bytes	1.0 GiB
Percentage	Proportion of total capacity	50%	Used for easier understanding.

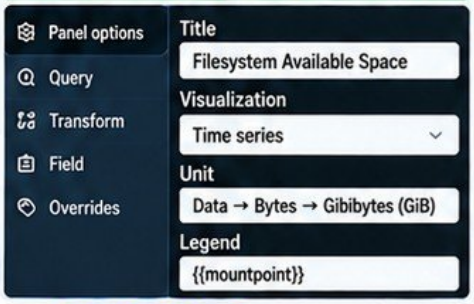
4. Common Disk Panel Queries

Purpose	PromQL Query
Available space for a specific filesystem	`node_filesystem_avail_bytes{mountpoint="/"}`
Total space for a filesystem	`node_filesystem_size_bytes{mountpoint="/"}`
Used space (bytes)	`node_filesystem_size_bytes{mountpoint="/"} - node_filesystem_avail_bytes{mountpoint="/"}`
Usage percentage	`100 * (1 - (node_filesystem_avail_bytes{mountpoint="/"} / node_filesystem_size_bytes{mountpoint="/"}))`

5. Grafana Memory Panel Settings



6. Grafana Disk Panel Settings



7. Important Considerations

- Always identify the filesystem (e.g. /, /var, /home) in the panel title or legend.
- Use the correct unit (GiB) for readability.
- Filter out unnecessary filesystems like tmpfs, overlay, or container-specific mounts if you only want real disk usage.
- Consider showing both available space and usage percentage.

8. Beginner Mistakes

- Adding all filesystem metrics together (leads to incorrect totals).
- Including tmpfs, overlay, or container mount points.
- Not filtering by mountpoint, so the panel shows many unnecessary lines.
- Using the wrong unit (bytes instead of GiB).
- Not showing which filesystem is being measured.

9. Practice

1. Create a memory available panel and display it in GiB.
2. Create a disk space panel for the / filesystem.
3. Add a second disk panel for /var (optional).
4. Observe the values and verify they match your server.
5. Try filling your disk with a large file (in a safe way) and watch the graph change, then delete the file.

10. Key Takeaway

A memory and disk panel is useful only when it clearly shows what is being measured (filesystem, unit, and meaning). Always filter and label your metrics so anyone looking at the dashboard can understand the server's health.

VERIQTA

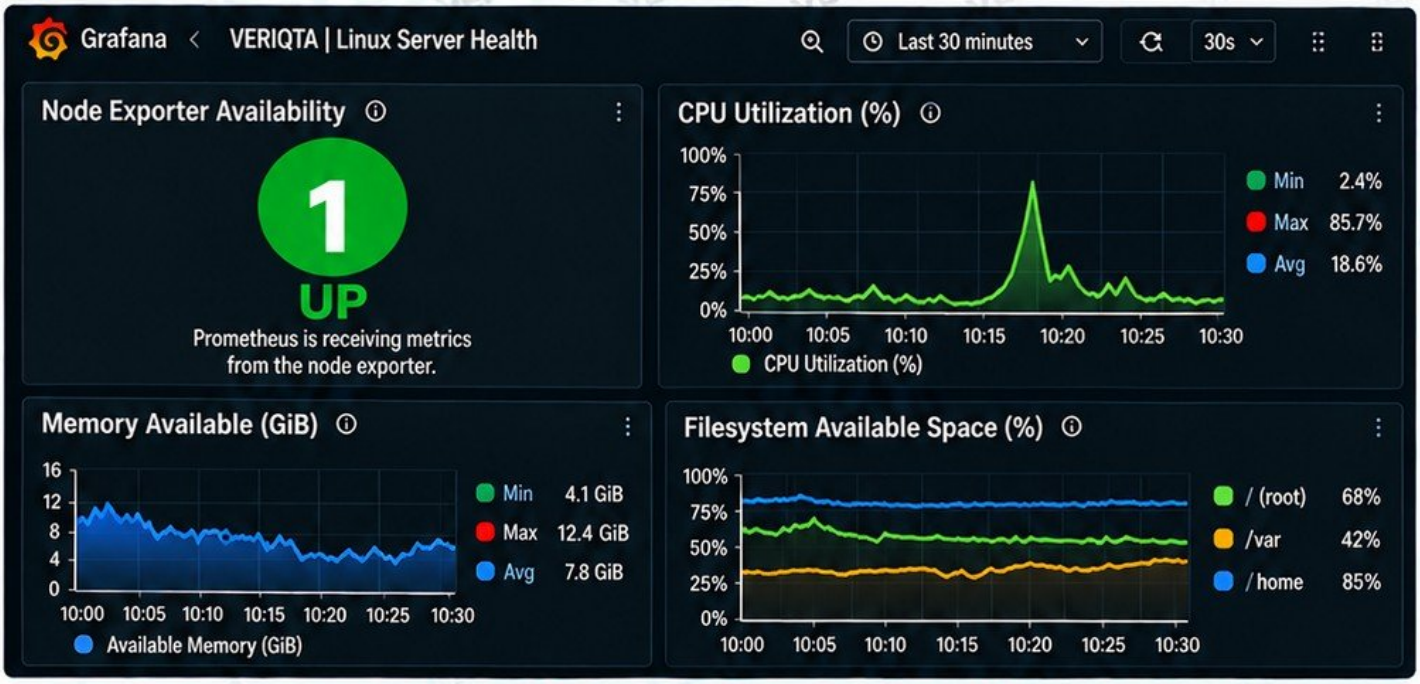
Make Your Dashboard Useful During an Incident

17

Key Idea

During an incident, engineers need a clear, simple view of the most important information so they can quickly understand what is happening and decide the next step.

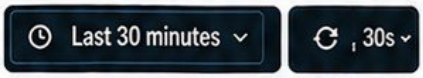
1. Complete VERIQTA Dashboard (Four Panels)



2. Organize Panels for Fast Understanding

1. Place the most important information at the top left (e.g. target availability).
2. Put CPU utilization next to it for quick performance insights.
3. Add memory availability below.
4. Add filesystem space next to memory.
5. Use clear titles so anyone can understand what each panel shows.

3. Choose Time Range and Refresh Rate



- Use a short time range (e.g. Last 30 minutes) during an incident.
- Set a fast refresh interval (e.g. 10s or 30s).
- You can use a longer time range for trends (e.g. Last 6 hours or Last 24 hours) after the incident.

4. Configure Useful Options

1. Set clear panel titles (e.g. CPU Utilization (%)).
2. Use appropriate units (%, GiB).
3. Add legends so you know what each line means.
4. Use helpful thresholds and colors (green = healthy, yellow = warning, red = investigate) for your environment.
5. Keep the dashboard simple and focused on key metrics.

5. Example Panel Thresholds (Adjust for Your Environment)

Metric	Normal (Green)	Warning (Yellow)	Critical (Red)	Note
CPU Utilization (%)	< 70%	70 – 85%	> 85%	Adjust based on your workload.
Memory Available (GiB)	> 2 GiB	1 – 2 GiB	< 1 GiB	Depends on total RAM and usage.
Filesystem Available (%)	> 20%	10 – 20%	< 10%	Different filesystems may have different thresholds.

6. Why Thresholds Help

- Make it easier to spot potential problems.
- Help you focus on what needs investigation.
- Give quick visual context (healthy, warning, critical).
- Do not assume the same thresholds for every system.
- Adjust thresholds based on your environment and requirements.

7. Practice

1. Create the complete dashboard with the four panels.
2. Set the time range to Last 30 minutes.
3. Set the refresh interval to 30s.
4. Configure units, legends, and thresholds.
5. Generate some CPU activity and watch the panels update.

8. Common Mistakes

- Using the wrong units (bytes instead of GiB or %).
- Not showing which filesystem is being measured.
- Setting thresholds that are too strict or too loose.
- Filling the dashboard with too many panels.
- Using a long refresh interval during an incident.

9. Key Takeaway

A useful dashboard during an incident shows the most important information clearly, with the right time range, refresh rate, units, legends, and sensible thresholds so engineers can quickly understand the system and take action.

VERIQTA

Break Monitoring on Purpose: Target DOWN

18

Key Idea

- up = 0 means Prometheus could not successfully scrape the target.
- It does not automatically mean the entire server is down.
- Always investigate to find the real cause.

1. Normal State: Target UP

Node Exporter Availability

1

UP

Prometheus is receiving metrics from the node exporter.

2. Stop the Node Exporter Container

1 Stop the node exporter using Docker Compose.

```bash
docker compose stop node-exporter
```

2 Check the container status.

```bash
docker compose ps
```

NAME	COMMAND	STATE	PORTS
prometheus	"/bin/prometheus"	running	0.0.0.0:9090->9090/tcp
grafana	"/run.sh"	running	0.0.0.0:3000->3000/tcp
node-exporter	""	exited	

3. Target DOWN in Grafana

Node Exporter Availability

0

DOWN

Prometheus is not receiving metrics from the node exporter.

4. Check Prometheus Targets

Prometheus Alerts Graph Status Help

Targets

All (2) Healthy (1) Unhealthy (1)

Endpoint	State	Labels	Last Scrape	Error
http://prometheus:9090/metrics	UP	job="prometheus"	3.2s ago	-
http://node-exporter:9100/metrics	DOWN	job="node"	15.6s ago	context deadline exceeded

5. Investigate the Failure

1 Check container logs.

```bash
docker compose logs node-exporter
Error: listen tcp :9100: bind: address already in use
Server exiting
```

2 Test the metrics endpoint.

```bash
curl http://localhost:9100/metrics
curl: (7) Failed to connect to localhost port 9100:
Connection refused
```

6. Recovery: Start the Node Exporter

1 Start the container.

```bash
docker compose start node-exporter
node-exporter  started
```

2 Check the status again.

```bash
docker compose ps
```

NAME	COMMAND	STATE	PORTS
node-exporter	"/bin/node_exporter"	running	0.0.0.0:9100->9100/tcp

3 Verify the metrics endpoint.

```bash
curl http://localhost:9100/metrics
# HELP node_cpu_seconds_total ...
# TYPE node_cpu_seconds_total counter
node_cpu_seconds_total{..} 12345.67
```

7. Target Back UP

Node Exporter Availability

1

UP

Prometheus is receiving metrics from the node exporter.

8. What Happened?

- Prometheus tried to scrape http://node-exporter:9100/metrics.
- The node exporter was stopped, so Prometheus could not connect.
- The target state changed from UP to DOWN.
- The Grafana panel showed 0 (DOWN).
- After starting the container, Prometheus could scrape again and the target returned to UP.

9. Key Takeaways

- up = 0 means the target is not being scraped successfully.
- It does not always mean the whole server is down.
- Check container status, logs, and the metrics endpoint.
- A single target can be down while other services are still working.
- Always investigate the cause.

10. Common Causes

- Container stopped or crashed.
- Wrong port or configuration.
- Network/Docker network issue.
- Firewall blocking the port.
- Wrong target address in prometheus.yml.
- Node exporter running but unreachable from Prometheus.

Why a Healthy Dashboard Is Not Enough

Key Idea

- A dashboard shows you what is happening.
- Alerts tell you when something is wrong.
- Someone must respond to investigate and fix the issue.

1. Metrics, Dashboards, and Alerts: What's the Difference?

Metrics

- Numeric data collected over time (e.g. CPU, memory, request rate).
- Stored in Prometheus.
- Used to answer questions like "What is happening?"

Example:
`node_cpu_seconds_total`,
`node_memory_MemAvailable_bytes`

Dashboards

- Visualize metrics with graphs, stat panels, and tables.
- Help you quickly understand the current state.
- Used to explore and investigate the system.

Example:
Grafana dashboard showing CPU, memory, disk, and application metrics.

Alerts

- Use rules to detect when a metric crosses a condition.
- Run continuously in Prometheus (Alerting).
- Send notifications (e.g. to email, Slack, Telegram, PagerDuty).
- Used to tell you "Something needs attention now."

Example:
CPU > 85% for 5 minutes
→ send an alert.

2. How Monitoring and Alerting Work Together

1. Metric



System metrics are collected by exporters and stored in Prometheus.

2. Alert Rule



A rule checks if a metric meets a condition for a set period of time.

3. Notification



If the condition is true, a notification is sent to the right channel (e.g. email, Slack, Telegram).

4. Engineer



An engineer receives the alert, investigates the issue, and takes action.

3. Alert Rule Basics

Part	What It Means
Condition	The rule that must be true (e.g. CPU > 85%).
Evaluation period	How long the condition must be true (e.g. for 5 minutes).
Labels	Extra information (e.g. severity = critical).
Annotations	Human-readable message with details.
Notification routing	Where the alert is sent (email, Slack, Telegram, etc.).

4. Example Alert Rule (CPU High)

PromQL (Alert condition)

```
100 * (1 - avg(rate(node_cpu_seconds_total
{mode="idle"}[5m]))) > 85
```

Meaning	CPU utilization is above 85%.
Evaluation period	For 5 minutes.
Severity	critical
Notification	Send to your chosen channel (e.g. Telegram).

5. Monitoring Needs More Than a Dashboard

- A dashboard can look healthy while the application is still broken.
- You also need application health checks (e.g. /health endpoint).
- Logs help you understand what is happening.
- Traces help you follow requests across services.
- Monitoring is not complete without an incident response process and someone to respond to alerts.

Production Insight

- A healthy dashboard does not mean everything is working correctly.
- Metrics only show part of the picture.
- Use metrics, logs, traces, application health checks, and real incident response.
- The goal is not just to see problems, but to detect, investigate, and fix them quickly.

7. Common Mistakes

- Thinking a green dashboard means there are no problems.
- Not setting up alerts.
- Using alert thresholds that are too sensitive.
- Not routing alerts to the right people.
- No clear plan for who responds.

8. Key Takeaway

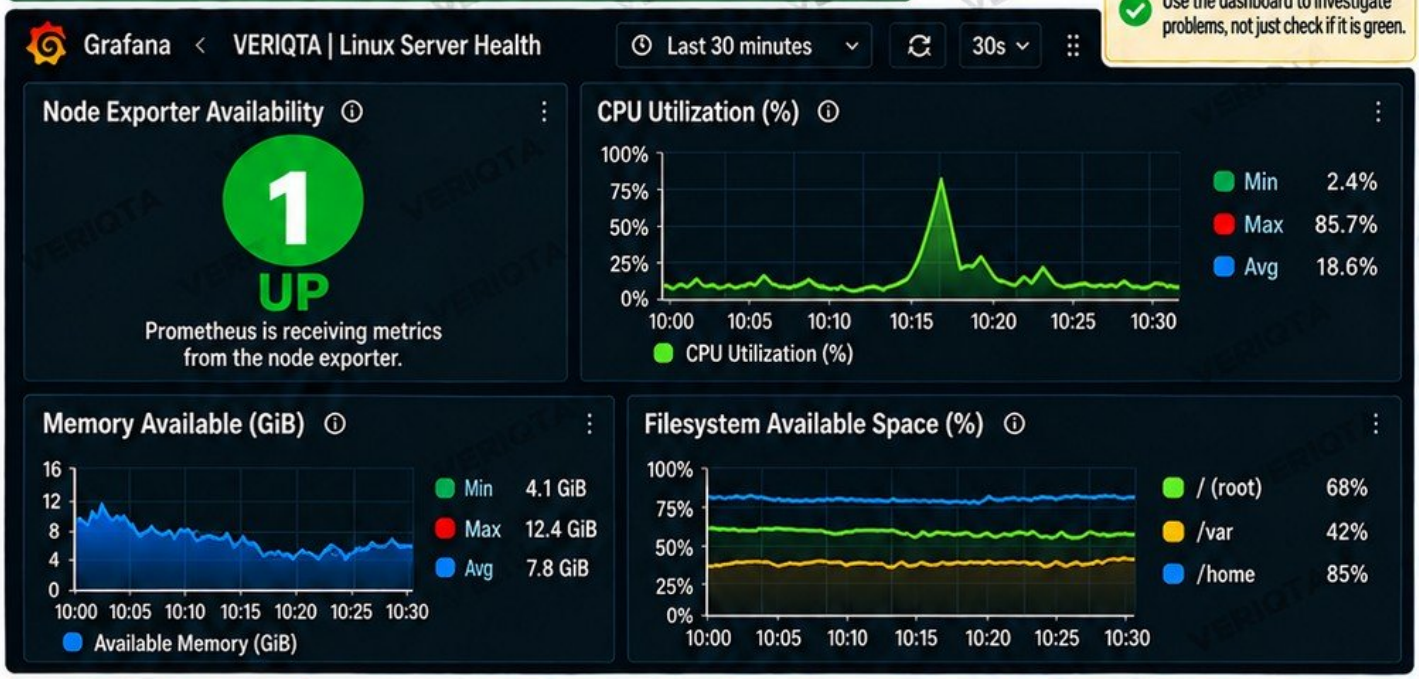
Dashboards show you the current state, but alerts and a response process turn monitoring into real reliability.

Final Project: Can You Trust Your Dashboard?

Project Goal

- Build a real monitoring dashboard.
- Verify live data and understand every panel.
- Break and recover the target.
- Use the dashboard to investigate problems, not just check if it is green.

1. Completed VERIQTA | Linux Server Health Dashboard



2. Your Challenge

1. Verify that all four panels show live data.
2. Explain what each panel means.
3. Stop the Node Exporter container.
4. Observe the dashboard and Prometheus target state.
5. Investigate the failure using the right tools.
6. Start the Node Exporter container again.
7. Confirm that the dashboard returns to normal.
8. Save the finished dashboard.

3. Stop and Restart the Node Exporter

1 Stop the container

```bash
docker compose stop node-exporter
```

The Grafana target availability panel should change from UP to DOWN.

2 Start the container

```bash
docker compose start node-exporter
```

The Grafana target availability panel should return from DOWN to UP.

4. Investigate the Failure

1. Check container status.

```bash
docker compose ps
```

2. Check container logs.

```bash
docker compose logs node-exporter
```

3. Check the metrics endpoint.

```bash
curl http://localhost:9100/metrics
```

5. What to Look For in Each Panel

Panel	What It Shows	What to Check
Node Exporter Availability	Whether Prometheus is scraping the target (up = 1).	Should be 1 (UP). If 0 (DOWN), investigate why.
CPU Utilization (%)	Host CPU usage over time (from CPU idle time).	Check for normal levels, spikes, and sustained high usage.
Memory Available (GiB)	Available memory on the server.	Verify the value is reasonable and not close to zero.
Filesystem Available (%)	Available disk space for selected filesystems.	Check the correct filesystem (e.g. /, /var, /home) and look for low space.

6. Final Question

If a graph stops updating, is the server unhealthy, the exporter broken, or the monitoring connection failing?

What evidence would distinguish them?

Possible Evidence to Check

- Container status and logs.
- Prometheus target state (UP or DOWN).
- Metrics endpoint (curl).
- Other services on the server.
- Logs, application health checks, or traces.

7. Common Mistakes

- Assuming a green dashboard means everything is working.
- Not checking the Prometheus target page.
- Not investigating when a graph stops updating.
- Confusing server issues with exporter or network issues.
- Not verifying the correct filesystem or units.

8. Key Takeaway

- A dashboard is useful only when you understand what it is showing.
- When something looks wrong, investigate and find the real cause.
- Monitoring is a tool that helps you detect and solve problems, not a guarantee that nothing will fail.